

CODE3D-AI

Complete Backend Architecture & Engineering Concept Guide

Comprehensive Technical Analysis: KON, KAISE, AUR KYU KAAM KAR RAHA HAI

Includes: Dual-Backend Architecture (Spring Boot & Node.js), 5-Language AST Sandbox, Neon DB & Local Fallback, AI Tutor Engine

EXECUTIVE OVERVIEW (SAMIKSHA)

- CODE3D-AI ek dual-engine full-stack platform hai jo normal compiler ki tarah sirf output nahi deta, balki code ki har ek line ke execution trace (variables, arrays, memory frames) ko real-time 3D WebGL me visualize karwata hai.
- Backend do alag-alag powerful engines par divide kiya gaya hai: Node.js (Port 5000) for fast sandbox API & microservices, aur Spring Boot (Port 8080) for deep Java AST parsing and complex DSA algorithm pattern engines.
- Ye document explains karta hai ki kaun se components backend me hain (KON), unka internal workflow step-by-step kaise kaam karta hai (KAISE), aur unhe is tarah kyu design kiya gaya hai (KYU).

1. SYSTEM ARCHITECTURE & BIG PICTURE

Kaun kaun se servers aur databases sath me integrate hain

CODE3D-AI ka architecture 4 core tiers me bata hua hai. Ye traditional monolithic app nahi hai balki ek multi-engine hybrid architecture hai jo educational reliability aur real-time rendering ke liye tailor-made hai:

Component	Port & Tech Stack	Primary Role & Responsibility
Frontend Studio	Port 5173 React 18, Three.js, Monaco	User code input, 3D visualization canvas, playback timeline (0.25x - 4x)
Node.js Engine	Port 5000 Express, Prisma, Sandboxes	API routing, 5-Language Sandbox (JS, Py, Java, C++, C), JWT Auth, AI Tutor
Spring Boot API	Port 8080 Java 21, JavaParser, JPA	Deep Java AST compilation, complex DSA catalog (Heap, LRU, Trie, Graph)
Neon PostgreSQL	Cloud Hosted (AWS us-east-2)	Permanent storage for users, saved codes, execution histories, quizzes
Local Fallback	local_db.json + In-Memory Arrays	Zero-downtime offline fallback: DB offline hone par bhi app 100% chalti hai

2. KON: KAUN KAUN SE COMPONENTS HAIN?

Detailed breakdown of Node.js and Spring Boot backend actors

A. Node.js Express Engine (server/)

Node.js server port 5000 par run karta hai aur light-weight, event-driven API requests aur multi-language execution trace synthesis ko handle karta hai. Iske core modules:

Key Modules in Node.js Engine (server/src)

- app.js & index.js: Express application setup, Helmet security headers, CORS origin allowlist, rate-limiting, aur JSON/cookie parsing.
- sandbox/executionEngine.js: Main execution coordinator. Code size (50KB limit), regex security scanning, timeout race (5000ms), aur step limits (10,000 steps cap) manage karta hai.
- adapters/traceAdapters.js: Har language ke liye dedicated state-machine tracer hai jo code ko line-by-line parse karke 3D step events produce karta hai.
- controllers/authController.js: User signup, login, bcrypt hash generation (10 rounds), aur 32-byte crypto session tokens banata hai.
- controllers/executionController.js: /api/execute route par sandbox execute karta hai aur results ko database ya memory me save karta hai.
- controllers/aiController.js: Google Gemini 1.5 Flash API ke sath integrate hokar code aur variables ka live DSA analysis generate karta hai.
- db.js & localStore.js: Hybrid database client jo pehle Neon cloud connect karta hai, fail hone par automatic JSON file me fallback ho jata hai.

B. Spring Boot Backend (backend/)

Java 21 aur Spring Boot 3 par bana ye engine port 8080 par serve hota hai. Iska primary use deep static analysis, true Java AST parsing, aur enterprise data management hai:

Key Modules in Spring Boot Backend (backend/src/main/java/com/code3d)

- **JavaAstExecutionEngine.java:** JavaParser library use karke true AST (Abstract Syntax Tree) generate karta hai. Ye MethodDeclaration, ForStmt, IfStmt, WhileStmt ko simulate karta hai aur variable scope track karta hai.
- **MultiLanguageExecutionService.java:** 3500+ lines ka massive engine jo trapping Rain Water, LRU Cache, Trie, Heap, DSU, LIS jaise complex algorithms ke custom visual steps synthesize karta hai.
- **UniversalCodeAnalyzer.java & JavaCodeAnalyzer.java:** Static code analysis, syntax errors detection, aur asymptotic Big-O time complexity calculate karta hai.
- **Repository Layer:** Spring Data JPA (UserRepository, ProgramRepository, ExecutionRepository, QuizRecordRepository) io HikariCP pooling ke sath Neon PostgreSQL se directly link karta hai.
- **CorsConfig.java:** Cross-Origin Resource Sharing configure karta hai taaki frontend Vite (port 5173) bina kisi security block ke API hit kar sake.

3. KAISE: YEH SAB KAISE KAAM KAR RAHA HAI?

End-to-End lifecycle of code execution, authentication, and fallback

Step-by-Step Code Execution Pipeline (The Core Loop)

Jab user Monaco Editor me code likh kar "Run / Execute" dabata hai (Ctrl + Enter), backend me ye 7 steps sequentially execute hote hain:

Lifecycle: Code to 3D Visualization

- **Step 1 (Frontend Dispatch):** ExecutionManager.js check karta hai ki preferBackend flag true hai ya nahi. Agar true hai, toh HTTP POST request api/execution/run par payload bhejta hai: { code, language, input, title }.
- **Step 2 (Security Gate Check):** server/src/sandbox/executionEngine.js sabse pehle regex blacklist check karta hai. Agar code me "process.exit", "child_process", "fs", "System.exit", "Runtime.getRuntime" jaisi koi malicious command mili hai, toh execution turant block ho jati hai.
- **Step 3 (Adapter Selection):** getAdapter(language) call hota hai (e.g., JavaScriptTraceAdapter, PythonTraceAdapter, JavaTraceAdapter).
- **Step 4 (Trace Generation):** Selected adapter code ko parse karta hai. Har loop iteration, variable assignment, aur array mutation par ek normalized ExecutionStep create hota hai.
- **Step 5 (Timeout & Limit Race):** Promise.race() lagaya jata hai (5000ms max timeout). Agar code infinite loop me phas jaye (jaise while(true)), toh server hang nahi hota balki 5 second me graceful TIMEOUT error return karta hai.
- **Step 6 (Database Auto-Persist):** executionController.js result ko check karta hai. Agar Neon database online hai, toh Prisma ya JPA se execution history save hoti hai. Agar DB offline hai, toh memory store me save hoti hai.
- **Step 7 (3D Presentation in Frontend):** Response JSON format me frontend ko mita hai. Frontend ka TimeMachineScrubber aur Three.js Mesh engine har step ke "dataStructureState" ko canvas par 3D cubes aur glowing pointers ke roop me animate kar deta hai.

Data Contract: ExecutionStep Structure

Backend jo JSON array return karta hai, usme har step ka standard structure ye hota hai:

NORMALIZED EXECUTIONSTEP JSON FORMAT

```
{
  "stepNumber": 4,
  "lineNumber": 12,
  "eventType": "ARRAY_MUTATE",
  "variables": { "i": 2, "temp": 45, "arr": [12, 34, 45, 90] },
  "changedVariable": "arr",
  "dataStructureState": {
    "type": "array",
    "values": [12, 34, 45, 90],
    "activeIndex": 2,
    "pointers": { "i": 2 }
  },
  "explanation": "Updated arr[2] to 45. Pointer i incremented to 2."
}
```

Authentication & Security Flow (Kaise Login/Signup Kaam Karta Hai)

User security ke liye backend stateless token aur cookie mechanism follow karta hai:

Authentication Lifecycle

- **Registration:** User username, email aur password bhejta hai. authController.js password ko bcryptjs ke sath 10 salt rounds me hash karta hai. Plaintext password kabhi database me save nahi hota.
- **Session Token:** crypto.randomBytes(32).toString('hex') se 64-character unguessable cryptographic token generate hota hai.
- **Dual Delivery:** Token ko HTTP-Only cookie ("code3d_session") me set kiya jata hai (taaki XSS attack se bacha ja sake) aur JSON response me bhi diya jata hai taaki mobile/API clients Bearer header use kar sakein.
- **RequireAuth Middleware:** Har protected route par auth.js check karta hai ki cookie ya Authorization header me token valid hai ya nahi. Agar database down hai, toh local memory session list se verify karta hai.

Zero-Downtime Database & Fallback Flow (Kaise DB Resilience Kaam Karta Hai)

Database layer ko is tarah program kiya gaya hai ki agar internet na ho ya Neon Cloud down ho, tab bhi app 100% smoothly chale:

Database Fallback Logic

- 1. `checkDatabaseConnection()`: Server start hote hi "SELECT 1" query execute karta hai Neon PostgreSQL par.
- 2. If Online: `isDbOnline()` flag ko true mark karta hai. Saare users, code snippets, aur histories Neon DB me Prisma / JPA ke through persist hote hain.
- 3. If Offline: Catch block trigger hota hai. Server crash hone ke bajay `isDbOnline()` ko false mark karta hai aur automatic `localStorage` (data/local_db.json) aur in-memory arrays me shift ho jata hai.
- 4. Result: Student ya developer ko kahi red error screen nahi aati, application gracefully operate karti rehti hai.

4. KYU: YEH ARCHITECTURE KYU CHUNA GAYA?

Architectural decisions, rationale, and design tradeoffs

1. Kyu AST & Trace Steps (Normal Terminal Execution Kyu Nahi?)

Traditional platforms (jaise LeetCode, HackerRank) code ko compile karke sirf final output (stdout) return karte hain. Lekin CODE3D-AI ka mission hai: "Don't just read the code, SEE the code execute." 3D canvas me ek ek bar ko animate karne ke liye, algorithm ke har intermediate step ki exact memory state chahiye hoti hai (kaunsa index highlight hoga, swap kaise hoga, recursion frame kab push/pop hua). Isliye backend har instruction ko parse karke granular time-series trace emit karta hai.

2. Kyu Do Backends (Node.js + Spring Boot)?

- Node.js: Super fast lightweight non-blocking I/O server hai. Ye JavaScript aur Python parsing, Express routing, WebSockets, aur rapid local development ke liye best hai.
- Spring Boot: Enterprise-grade Java 21 platform hai. JavaParser library world-class static AST analysis provide karti hai jo JavaScript me achieve karna bahut complex aur error-prone hota. Saath hi Spring Boot multi-language DSA catalog (Heap, Trie, DSU, LRU) ke 3,500+ lines of algorithmic simulation ko strongly-typed tarike se handle karta hai.

3. Kyu Strict Sandboxing & Limits?

Agar user ko server par arbitrary code execute karne diya jaye bina sandbox ke, toh koi bhi malicious user "rm -rf /", infinite memory allocations (fork bombs), ya ".env" file chura sakta hai. Isliye:

- 5000ms Execution Timeout: Infinite while loop se bachaane ke liye.
- 10,000 Step Cap: Browser ya server memory overflow na ho.
- Regex Security Guard: File system (fs), ProcessBuilder, aur Child Processes ko block karne ke liye.

4. Kyu Hybrid Database Fallback?

Colleges, hackathons, ya travel ke dauran internet connectivity hamesha stable nahi hoti. Agar platform cloud database par 100% tightly coupled hota, toh network issue aate hi pura editor ruk jata. Hybrid fallback model ensures karta hai ki student offline laptop par bhi bina kisi error ke har data structure aur algorithm ko simulate kar sake.

5. API ENDPOINTS & CONTRACTS CHEATSHEET

Quick reference of all REST endpoints exposed by the backends

Method & Endpoint	Backend Engine	Purpose / What It Does
POST /api/executions/run	Node.js (5000)	Executes source code in 5 languages & returns 3D trace array
POST /api/execute	Spring Boot (8080)	JavaParser AST execution with dynamic variables & Scanner support
POST /api/analyze	Node.js / Spring	Static code analysis, loop nesting depth & Big-O complexity estimation
POST /api/auth/register	Node.js & Spring	Registers user, hashes password with bcrypt, issues session token
POST /api/auth/login	Node.js & Spring	Validates credentials, sets HttpOnly cookie & returns user profile
GET /api/auth/me	Node.js & Spring	Validates current active session and returns user info
GET /api/history	Node.js (5000)	Fetches previous execution traces with one-click replay support
POST /api/ai/explain	Node.js (5000)	Sends line, variables, callstack to Gemini 1.5 Flash for AI explanation
GET /api/dsa/topics	Node.js (5000)	Returns catalog of DSA topics (Arrays, Trees, Graphs, DP, etc.)
GET /api/quiz/attempts	Node.js (5000)	Retrieves user quiz scores and algorithmic mastery stats
GET /api/health	Both (5000/8080)	Checks server health, DB connection status, and supported languages

CONCLUSION & SUMMARY

- CODE3D-AI ka backend sirf ek simple CRUD API nahi hai, balki ek custom-engineered State-Machine Simulator aur AST Interpreter hai.
- Ye architecture educational technology ke highest standards ko meet karta hai: Zero-downtime offline fallback, strict multi-layer security sandbox, aur millisecond-level granular trace generation.
- File generated on: 7/10/2026 | Target: CODE3D-AI Full-Stack Architecture

